

Git introduction

This document shall serve as both kickstart and reference for beginners. Though even experienced users may learn something new.

- Git instructions
 - [Git & Github](#)
 - [Git help](#)
 - [Configuration](#)
 - [Creating a repository](#)
 - [The concept of commits](#)
 - [Git Terminology](#)

Git & Github

Some might be confused about Git and Github. They have similar names, with Github just adding a 'hub'.

- **Git** is a version control system, that lets you save, analyse and edit the history of files. It can be used locally, or in combination with a server. The latter allows for collaboration with a team and can act as backup. Git is licensed under GPLv2.
- **Github** is a frontend for the Git server, which adds some functions like access control, pull requests or issues to name a few. It is not a usable service on its own. Github is proprietary and owned by Microsoft since 2018.

Git help

When unsure about a Git command append `--help`. This opens the most specific manual available for your command.

Example:

```
1 # show help for the 'config' subcommand
2 git config --help
3
4 # show help for the 'remote add' subcommand
5 git remote add --help
```

Configuration

Before starting to use Git it is recommended to configure the user name and email at the very least. Otherwise, Git will ask you at the first action which requires an identity, most probably a `push`.

Git looks for a file `~/.gitconfig` (`~` = users home directory). You can either (create and) edit the file directly, or use `git config`. See the example below for how to configure the user name and mail.

```
1 [user]
2     name = student
3     email = student@students.zhaw.ch
```

For Linux users keeping their configuration files in a repository, distributing them with symbolic links, the Git configuration can include files. So the user and email can be stored in a separate file.

```
1 [include]
2     path = /path/to/include/file
```

Sensible options to set are the editor used for commit messages, the default branch name and whether to merge or rebase. The value of editor is the launch command of the desired editor. Below is an excerpt from my configuration as an example.

```
1 [core]
2     editor = nvim
3
4 ; i do not care about politics, main is shorter than master
5 [init]
6     defaultBranch = main
7
8 [pull]
9     rebase = true
```

Creating a repository

When working with Git server (e.g. Github, Gitlab, Gitea, Forgejo, ..) there are two ways to start a repository:

- **Local:** Use `git init` to initialise the current directory as a Git repository. This will create a directory `.git` which contains the local information, e.g. history, commit hashes, hooks, etc. If you want to synchronise a local repository with a server, follow the steps for creating a remote repository and use `git remote add` to add a remote and its URL (default remote name is `origin`).

- **Remote:** Open the frontend of choice and create a new repository. On Github there is usually a green button in the top right corner. Choose a name, description and licence. Now get the URL to clone the repository locally, or set it as remote in an existing one. On Github click the green **Code** button and choose either **https** or **ssh** (recommended, but requires additional setup).

See examples for the Git commands mentioned in this section below.

```
1 # initialise directory as git repository
2 # use '.' as path to initialise current directory
3 git init <path/to/directory>
4
5 # add remote to local repository under name 'origin'
6 git remote add origin <url-to-remote-repository>
7
8 # change URL, for example to switch to ssh after cloning
9 # a repository over https
10 git remote set-url <remote-name> <remote-url>
11
12 # clone a remote repository
13 git clone <repository-url>
```

Choosing a licence is important, if only to deny liability for your code.

I recommend to setup a pair of ssh keys and use ssh connections to interact with the Git server. It is more secure and usually more convenient too.

The concept of commits

Before looking at how to add or change commits, it is prudent to get an understanding for how Git works.

A Git history is a linked list of changes, called commits. Each commit has one (or more for merge commits) parents and an arbitrary number of children. A commit is a delta relative to its parent(s) and does **not** contain the current files.

Git Terminology

Term	Description
head	pointer to commit currently checked out. on remote it will usually point to <i>main</i> .
index / staging area	<code>git add</code> puts file/directories (rather the changes made to them) into a cache, called <i>index</i> or <i>staging area</i> (in arguments it is also called <i>chache</i>). <code>git commit</code> adds the changes in that cache to the history with a hash, a descriptoin and a few more information about the commit.